

第十一章作业

题目

1. 试述事务的概念及事务的 4 个特性。数据库恢复技术能保证事务的哪些特性？

事务是用户定义的一个数据库操作序列，这些操作要么全部执行，要么全部不执行，是一个不可分割的工作单位。

事务具有 4 个特性，即 ACID 特性，包括：原子性（Atomicity）、一致性（Consistency）、隔离性（Isolation）和持续性（Durability）。

- 原子性：事务中的所有操作要么全部完成，要么全部不完成，不能只执行其中一部分。
- 一致性：事务执行前后，数据库必须从一个一致性状态转变到另一个一致性状态。
- 隔离性：多个事务并发执行时，一个事务内部的操作不能被其他事务干扰。
- 持续性：事务一旦提交，对数据库的修改就是永久性的，即使系统发生故障也不会丢失。

数据库恢复技术能够保证事务的原子性和持续性。

2. 为什么事务非正常结束时会影响数据库数据的正确性？请举例说明。

事务执行过程中，如果系统突然发生故障，事务可能只完成了一部分操作，而另一部分尚未执行，此时数据库中的数据就可能处于不一致状态。

例如，在银行转账系统中，事务 T 的操作为：

```
A账户余额 = A账户余额 - 1000
B账户余额 = B账户余额 + 1000
```

如果事务执行过程中系统突然断电，只完成了“A账户减 1000”而没有完成“B账户加 1000”，则数据库中的总金额会减少 1000，数据库状态出现错误。

因此，事务非正常结束会破坏数据库的一致性。

3. 登记日志文件时为什么必须先写日志文件，后写数据库？

数据库恢复依赖日志文件进行 REDO 和 UNDO 操作。

如果先修改数据库，而日志文件尚未写入，此时系统发生故障，则数据库已经改变，但日志中没有记录这些修改，系统将无法恢复这些数据。

如果先写日志，再写数据库，即使数据库尚未来得及修改，恢复时最多只会多执行一次 UNDO 操作，不会影响数据库的正确性。

因此，为了保证系统安全，必须遵循“先写日志文件，后写数据库”的原则。

4. 考虑表 11.2 所示的日志记录。

日志记录如下：

序号	日志
1	T1：开始
2	T1：写 A，A=10
3	T2：开始
4	T2：写 B，B=9
5	T1：写 C，C=11
6	T1：提交
7	T2：写 C，C=13
8	T3：开始
9	T3：写 A，A=8
10	T2：回滚
11	T3：写 B，B=7
12	T4：开始
13	T3：提交
14	T4：写 C，C=12

(1) 如果系统故障发生在序号 14 之后, 说明哪些事务需要重做, 哪些事务需要回滚。

已提交事务: T1、T3

未完成事务: T4

已回滚事务: T2

因此:

需要重做: T1、T3

需要回滚: T4

(2) 如果系统故障发生在序号 10 之后, 说明哪些事务需要重做, 哪些事务需要回滚。

已提交事务: T1

未完成事务: T3

T2 已回滚。

因此:

需要重做: T1

需要回滚: T3

(3) 如果系统故障发生在序号 9 之后, 说明哪些事务需要重做, 哪些事务需要回滚。

已提交事务: T1

未完成事务: T2、T3

因此:

需要重做: T1

需要回滚: T2、T3

(4) 如果系统故障发生在序号 7 之后, 说明哪些事务需要重做, 哪些事务需要回滚。

已提交事务: T1

未完成事务: T2

因此:

需要重做: T1

需要回滚: T2

5. 考虑表 11.2 所示的日志记录, 假设开始时 A、B、C 的值都为 0。

(1) 如果系统故障发生在序号 14 之后, 写出系统恢复后 A、B、C 的值。

T1 已提交, 需要 REDO:

A=10, C=11

T3 已提交, 需要 REDO:

A=8, B=7

T4 未提交, 需要 UNDO:

C 恢复为 11

因此最终结果为:

A=8, B=7, C=11

(2) 如果系统故障发生在序号 12 之后, 写出系统恢复后 A、B、C 的值。

T1 已提交, 需要 REDO。

T2 已回滚。

T3 尚未提交, 需要 UNDO。

因此恢复后:

A=10, B=0, C=11

(3) 如果系统故障发生在序号 10 之后, 写出系统恢复后 A、B、C 的值。

T1 已提交, 需要 REDO。

T2 已回滚。

T3 未提交, 需要 UNDO。

因此恢复后:

A=10, B=0, C=11

(4) 如果系统故障发生在序号 9 之后, 写出系统恢复后 A、B、C 的值。

T1 已提交, 需要 REDO。

T2、T3 均未提交, 需要 UNDO。

因此恢复后:

A=10, B=0, C=11

(5) 如果系统故障发生在序号 7 之后, 写出系统恢复后 A、B、C 的值。

只有 T1 已提交。

T2 未提交, 需要回滚。

因此恢复后:

A=10, B=0, C=11

(6) 如果系统故障发生在序号 5 之后, 写出系统恢复后 A、B、C 的值。

此时 T1 尚未提交, T2 也未提交, 因此两个事务都需要回滚。

恢复后:

A=0, B=0, C=0

6. 针对不同的故障类型（事务故障、系统故障、介质故障），试给出恢复的策略和方法。

（1）事务故障恢复

事务故障是指事务在执行过程中由于程序错误、死锁或人为中断等原因未能正常完成。

恢复步骤如下：

1. 反向扫描日志文件，找到该事务的更新记录。
2. 对事务执行 UNDO 操作。
3. 将数据库恢复到事务执行之前的状态。

（2）系统故障恢复

系统故障是指由于断电、操作系统崩溃等原因导致系统停止运行。

恢复步骤如下：

1. 正向扫描日志文件。
2. 找出已经提交的事务，加入 REDO 队列。
3. 找出未完成事务，加入 UNDO 队列。
4. 对 UNDO 队列中的事务执行回滚。
5. 对 REDO 队列中的事务重新执行。

（3）介质故障恢复

介质故障是指磁盘损坏等导致数据库文件丢失。

恢复步骤如下：

1. 装入最近一次数据库备份。
2. 装入备份之后的日志文件。
3. 利用日志文件执行 REDO 操作。
4. 将数据库恢复到故障前状态。

7. 什么是检查点记录？检查点记录包括哪些内容？

检查点记录是一种特殊的日志记录，用于记录数据库在某一时刻的状态，从而减少系统恢复时扫描日志的范围。

检查点记录主要包括以下内容：

1. 建立检查点时所有正在执行的事务列表。
2. 这些事务最近一个日志记录的地址。

通过检查点技术，数据库恢复时不必从日志开头扫描，可以从最近检查点开始恢复，提高恢复效率。

8. 具有检查点的恢复技术有什么优点？试举例说明。

使用检查点技术后，数据库恢复时只需要从最近一次检查点开始扫描日志，而不需要扫描全部日志，因此能够显著减少恢复时间。

例如，一个数据库系统已经运行了几个月，日志文件非常大。如果没有检查点，系统故障后需要从日志开头开始扫描，恢复时间会很长。

如果系统每隔一段时间建立一次检查点，那么恢复时只需要从最近检查点开始扫描日志即可，从而提高恢复效率。

此外，检查点还能减少不必要的 REDO 操作，降低系统恢复的开销。

9. 试述使用检查点方法进行恢复的步骤。

使用检查点恢复的方法如下：

1. 从重新开始文件中找到最近一次检查点记录的位置。
2. 根据检查点记录建立事务队列。

包括：

- UNDO-LIST：需要执行撤销操作的事务集合。
 - REDO-LIST：需要执行重做操作的事务集合。
3. 从检查点开始向后扫描日志文件。
 - 若发现事务提交，则将该事务移入 REDO-LIST。
 - 若事务未提交，则保留在 UNDO-LIST 中。
 4. 扫描结束后：
 - 对 UNDO-LIST 中事务执行 UNDO 操作。
 - 对 REDO-LIST 中事务执行 REDO 操作。
-

10. 什么是数据库镜像？它有什么用途？

数据库镜像是指数据库系统自动将数据库中的数据复制到另一个磁盘或存储设备中，并保持镜像数据与主数据库的一致性。

数据库镜像的主要用途包括：

- 1. 用于数据库恢复。当主数据库发生介质故障时，可以直接使用镜像数据库继续运行。
- 2. 提高数据库可用性。当主数据库正在被修改时，其他用户可以访问镜像数据库进行读取。
- 3. 提高系统可靠性。即使一个磁盘损坏，也不会导致数据完全丢失。

思维导图

